

Embedded Systems and IoT

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Docente: Prof. Alessandro Ricci

[Lab Notes]

ARDUINO/PC SERIAL COMMUNICATION

OBJECTIVE

- Learn how to enable communication between PC and Arduino through the serial port
- Tools and libraries
 - Arduino side: Serial library (+SoftwareSerial in future)
 - <http://arduino.cc/en/reference/serial>
 - PC side: Java language libs for serial communication
 - RXTX
 - http://en.wikibooks.org/wiki/Serial_Programming/Serial_Java
 - JSSC
 - java-simple-serial-connector
 - <https://code.google.com/archive/p/java-simple-serial-connector/>

SERIAL COMMUNICATION IN JAVA

- Original **JavaComm** Communication API spec
 - no reference implementations developed...
 - however: availability of open-source community libs
 - http://en.wikibooks.org/wiki/Serial_Programming/Serial_Java
- Among the most used:
 - **RxTx**
 - available on multiple platforms (Windows, Linux, Mac OS X)
 - implementing large part of the JavaComm spec
 - it requires to include a OS-dependent lib
 - Java Simple Serial Connector (**JSSC**)
 - similar to RxTx,
 - it does not require any OS-dependent lib to be explicitly included (just Java jars)

JSSC LIBRARY

- Lib web site
 - <https://github.com/java-native/jssc>
 - <https://github.com/java-native/jssc/releases>
 - <https://code.google.com/archive/p/java-simple-serial-connector/>
- To use it:
 - either include the `jssc.jar` in the classpath
 - or add the dependency using Gradle/Maven

RxTx LIBRARY

- Lib web site: <http://rxtx.qbang.org/wiki>
- To used it:
 - include the `RXTXcomm.jar` jar file in the class path
 - when launching Java programs, the specific OS lib must be linked
 - to this purpose, the path to the lib directory must be specified using the `java.library.path` property
 - example based on Linux:

```
java -cp rxtx-2.2-dist/linux-x64/RXTXcomm.jar  
      -Djava.library.path="rxtx-2.2-dist/linux-x64"  
      esiot.serial.ShowSerialPorts
```

REMARK: “:” in the class path is the separator for UNIX-based (Linux, Mac).
For Windows the separator is “;”

THE JAVACOMM SPEC

- Official docs: <http://www.oracle.com/technetwork/java/index-jsp-141752.html>
- Entry point class of the API: `CommPortIdentifier`
 - As a library, it provides useful functions for managing ports
 - `static java.util Enumeration getPortIdentifiers()`
 - `static CommPortIdentifier getPortIdentifier(String portName)`
 - As an object, it represents the identifier of a specific serial port
 - `CommPort open(java.lang.String appname, int timeout)`
 - open a communication session on the target port
 - the returned object represents the specific port opened (serial, parallel...)
 - There are different types of `CommPort`
 - serial ports are represented by the `SerialPort` class

SERIALPORT

- Doc:
 - http://docs.oracle.com/cd/E17802_01/products/products/javacomm/reference/api/javax/comm/SerialPort.html
- It provides methods to configure the serial protocol
 - `setSerialPortParams(int baudrate, int dataBits, int stopBits, int parity)`
 - ...
- It uses the Java **stream** model for sending/receiving data
 - `getInputStream()`
 - `getOutputStream()`
- Besides, the library provides the possibility to observe events that are generated by the serial port (pattern observer)
 - **SerialPortEventListener** interface
 - notify event: `serialEvent(SerialPortEvent or Event)`

SIMPLE SERIAL MONITOR

- Simple Java program that uses the serial port to communicate with Arduino, available in the repo
 - adapted from: <http://playground.arduino.cc/Interfacing/Java>
 - based on the Java Comm API
- Deployment
 - to be used with any Arduino program writing messages on the serial

JAVA EXAMPLES

- Example included in the repo
- **ShowSerialPorts**
 - printing in standard output the list of serial ports available
- **SimpleSerialMonitor**
 - implementing a simple serial monitor (like the ArduinoIDE one)
- **TestRemoteBlinking**
 - blinking example controlled by the PC
 - to be used with the Arduino program with `serial/remote_led_withmsg`
- **TestPingPong**
 - simple exchange of messages
 - to be used with the Arduino program `serial/remote_pingpong`

ARDUINO SIDE:

MSG-SERVICE LIBRARY

- A simple library (MsgService) for exchanging messages is included in the repo sources
- Class **MsgService** - lib with static methods for
 - Initialisation
 - void **init()**
 - check availability of messages:
 - bool **isMsgAvailable()**
 - msg fetching (no blocking)
 - Msg* **receiveMsg()**
- **Msg** class - representing a message
 - retrieval of the content as string: String getContent()

JAVA SIDE: SERIAL-COMM-CHANNEL LIBRARY

- `CommChannel` interface
 - basic methods for async message exchange
- `SerialCommChannel` implementation
 - msg queue to manage the messages
- Basic msg format
 - based on string